ViewShift: Dynamic Policy Enforcement for Every Data Lake

Walaa Eldin Moustafa

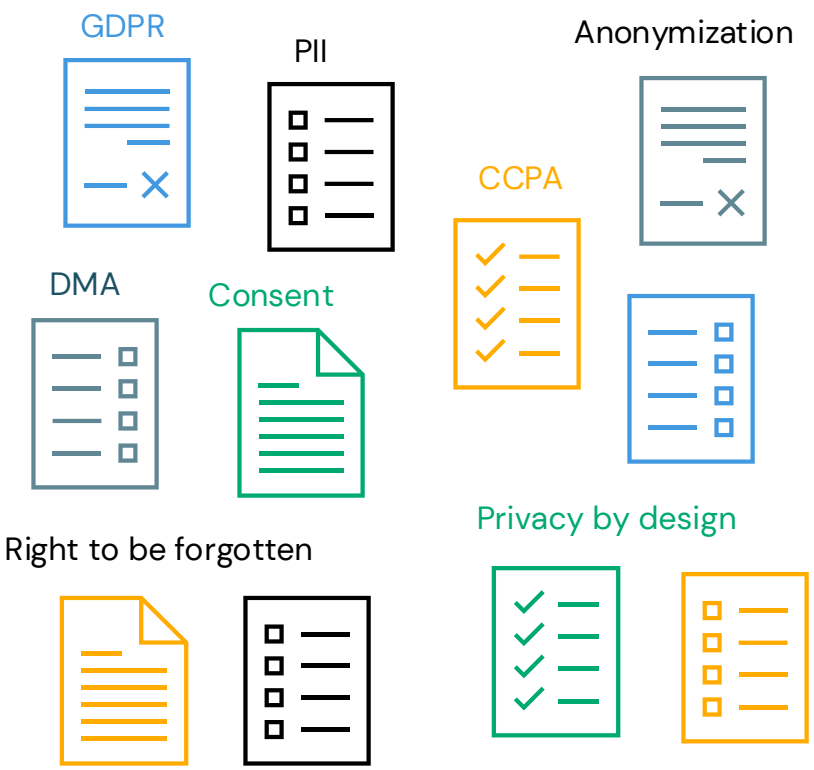
Khai Tran

June 10, 2025

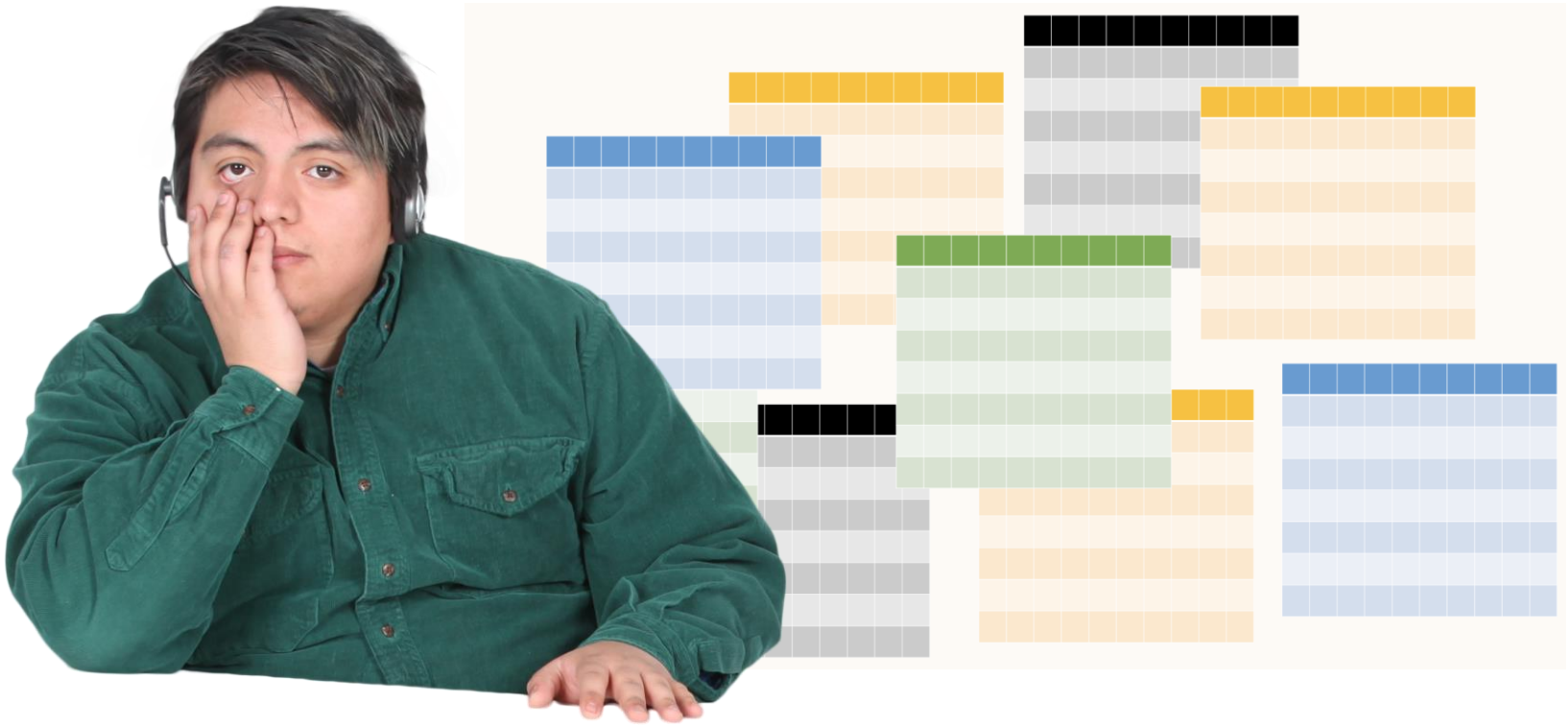
Data Protection Scene

Can you relate?

Too many policies

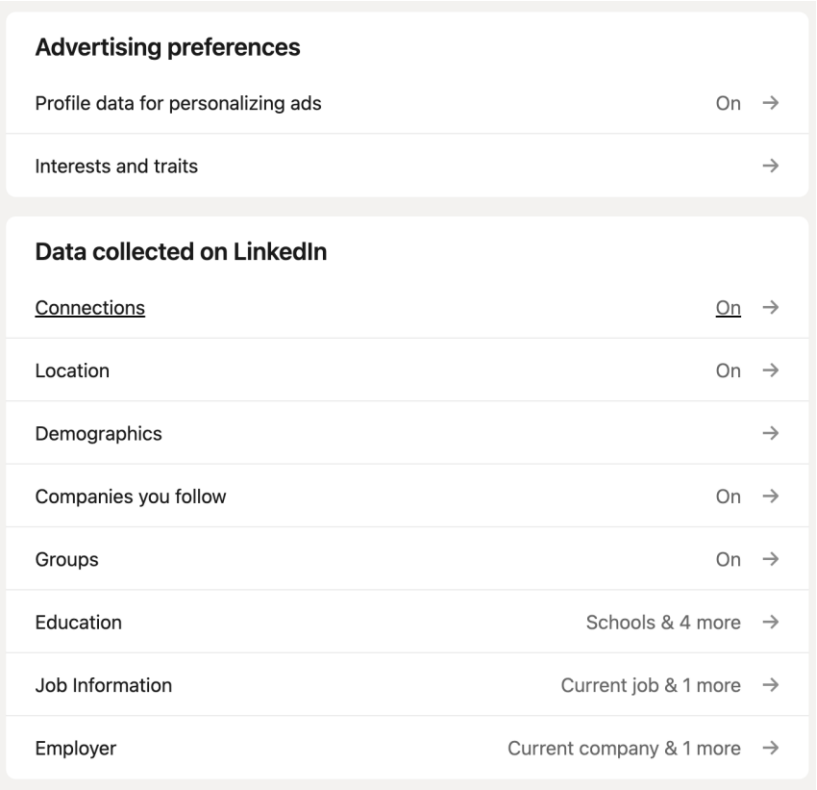


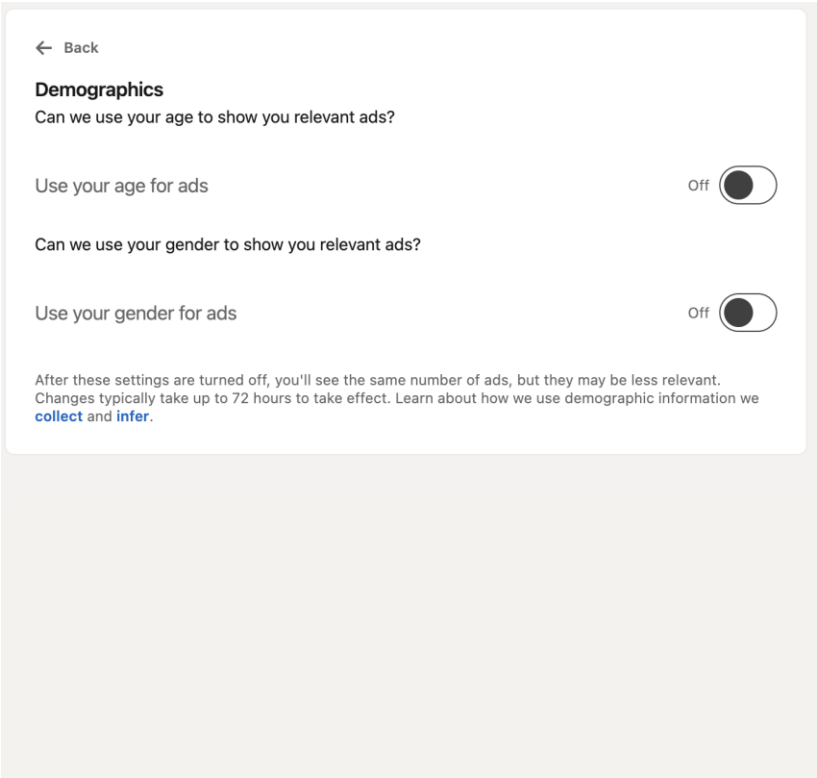
Too much data



The Rise of Privacy and Compliance

- Privacy Dashboards
- Security Checkups
- Data Export
- Data Deletion
- Ad Preferences





Settings

Solution is Easy!

Only 2 machines

Solution is Easy!

Only 2 machines

Policies

Data Lake Metadata

SQL
Views

Data & Applications

Compliance 🎉

Why Views

- **Expressive**
  - Express multiple policies with projections, filters, joins, UDFs.
- **Portable**
  - Executable on multiple engines.
- **Modular**
  - Can be drop-in replacement to underlying data
- **Agile**
  - Roll-out new views, rollback to previous views

```
CREATE VIEW T1_UC1 AS
SELECT
CASE WHEN consent = 'ALLOW'
THEN a ELSE obf(a)
FROM T1, Settings
WHERE Settings.ID = T1.ID
```

Why Views

Column level filtering

n	a	b	c	d	e
1	a1	b1	c1	d1	e1
2	a2	b2	c2	d2	e2
3	a3	b3	c3	d2	e3
4	a4	b4	c4	d4	e4
5	a5	b5	c5	d5	e5
6	a6	b6	c6	d6	e6
7	a7	b7	c7	d7	e7
8	a8	b8	c8	d8	e8
9	a9	b9	c9	d9	e9

n		b	c	d	e
1		b1	c1	d1	e1
2		b2	c2	d2	e2
3		b3	c3	d2	e3
4		b4	c4	d4	e4
5		b5	c5	d5	e5
6		b6	c6	d6	e6
7		b7	c7	d7	e7
8		b8	c8	d8	e8
9		b9	c9	d9	e9

Why Views

Column level masking

n	a	b	c	d	e
1	a1	b1	c1	d1	e1
2	a2	b2	c2	d2	e2
3	a3	b3	c3	d2	e3
4	a4	b4	c4	d4	e4
5	a5	b5	c5	d5	e5
6	a6	b6	c6	d6	e6
7	a7	b7	c7	d7	e7
8	a8	b8	c8	d8	e8
9	a9	b9	c9	d9	e9

n	a	b	c	d	e
1	#	b1	c1	d1	e1
2	#	b2	c2	d2	e2
3	#	b3	c3	d2	e3
4	#	b4	c4	d4	e4
5	#	b5	c5	d5	e5
6	#	b6	c6	d6	e6
7	#	b7	c7	d7	e7
8	#	b8	c8	d8	e8
9	#	b9	c9	d9	e9

Why Views

Row level filters

n	a	b	c	d	e
1	a1	b1	c1	d1	e1
2	a2	b2	c2	d2	e2
3	a3	b3	c3	d2	e3
4	a4	b4	c4	d4	e4
5	a5	b5	c5	d5	e5
6	a6	b6	c6	d6	e6
7	a7	b7	c7	d7	e7
8	a8	b8	c8	d8	e8
9	a9	b9	c9	d9	e9

n	a	b	c	d	e
1	a1	b1	c1	d1	e1
2	a2	b2	c2	d2	e2
4	a4	b4	c4	d4	e4
5	a5	b5	c5	d5	e5
6	a6	b6	c6	d6	e6
8	a8	b8	c8	d8	e8

Why Views

Cell level masking

n	a	b	c	d	e
1	a1	b1	c1	d1	e1
2	a2	b2	c2	d2	e2
3	a3	b3	c3	d2	e3
4	a4	b4	c4	d4	e4
5	a5	b5	c5	d5	e5
6	a6	b6	c6	d6	e6
7	a7	b7	c7	d7	e7
8	a8	b8	c8	d8	e8
9	a9	b9	c9	d9	e9

n	a	b	c	d	e
1	a1	b1	c1	d1	e1
2	a2	b2	c2	d2	e2
3	#	b3	c3	#	e3
4	a4	b4	c4	d4	e4
5	a5	#	c5	d5	#
6	a6	b6	c6	d6	e6
7	a7	b7	c7	d7	e7
8	#	b8	c8	d8	e8
9	a9	b9	#	#	#

Why Views

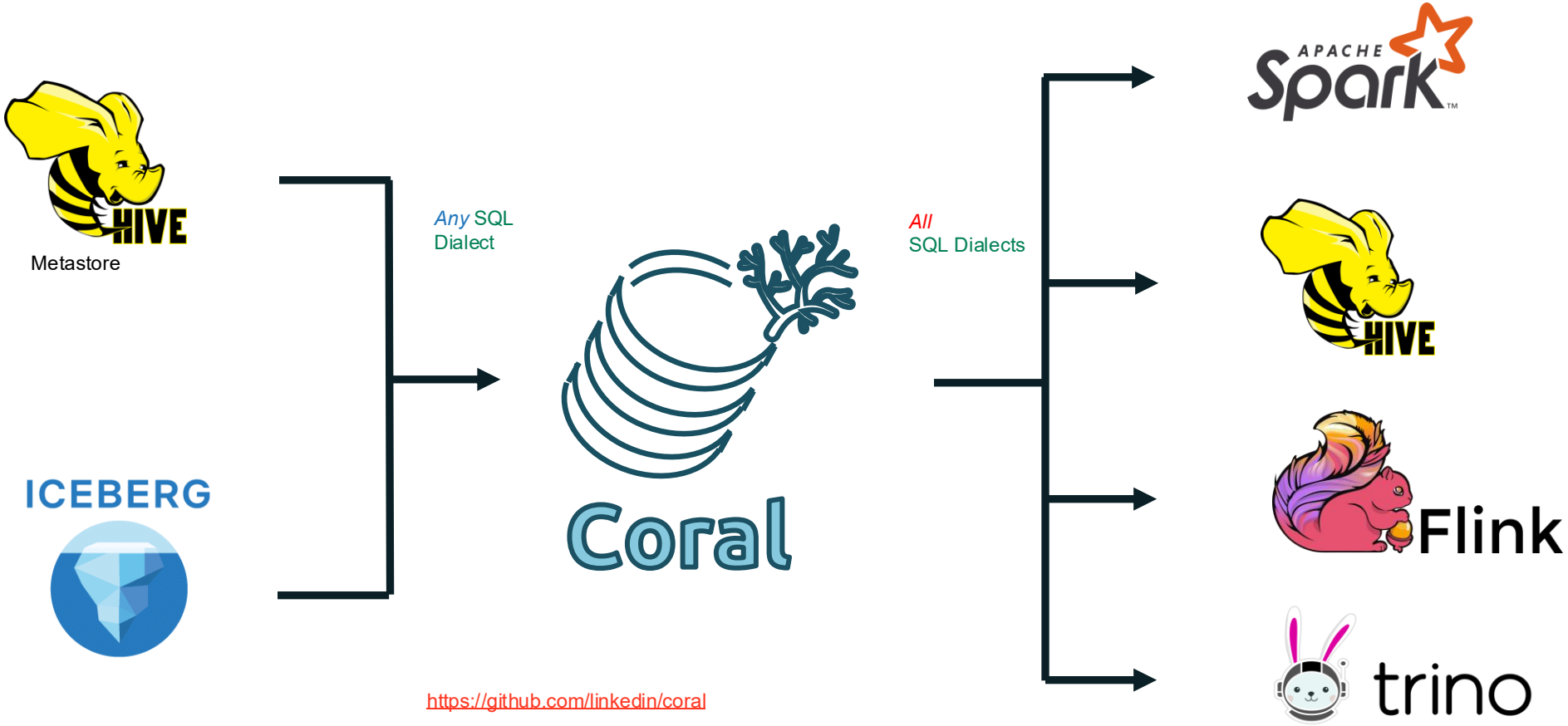
Multiple types of masking

n	a	b	c	d	e
1	a1	b1	c1	d1	e1
2	a2	b2	c2	d2	e2
3	a3	b3	c3	d2	e3
4	a4	b4	c4	d4	e4
5	a5	b5	c5	d5	e5
6	a6	b6	c6	d6	e6
7	a7	b7	c7	d7	e7
8	a8	b8	c8	d8	e8
9	a9	b9	c9	d9	e9

n	a	b	c	d	e
1	a1	b1	c1	d1	e1
2	a2	b2	c2	d2	e2
3	#	b3	c3	#	e3
4	a4	b4	c4	d4	e4
5	a5	#	c5	d5	#
6	a6	b6	c6	d6	e6
7	a7	b7	c7	d7	e7
8	#	b8	c8	d8	e8
9	a9	b9	#	#	#

Why Views

Multiple Query Engines, Multiple Metadata Sources



https://github.com/linkedin/coral

Compliance Views

Tables
T1
T2
T3
T4
T5

Metadata store

Compliance Views

Tables
T1
T2
T3
T4
T5

Views		
T1_UC1	T1_UC2	T1_UC3
T2_UC1	T2_UC2	T2_UC3
T3_UC1	T3_UC2	T3_UC3
T4_UC1	T4_UC2	T4_UC3
T5_UC1	T5_UC2	T5_UC3

Metadata store

Compliance Views

Tables
T1
T2
T3
T4
T5

Views		
T1_UC1	T1_UC2	T1_UC3
T2_UC1	T2_UC2	T2_UC3
		3_UC3
		4_UC3
T5_UC1	T5_UC2	T5_UC3

```
CREATE VIEW T1_UC1 AS
SELECT
CASE WHEN Settings.consent = 'ALLOW'
THEN a ELSE obf(a)
FROM T1, Settings
WHERE Settings.ID = T1.ID
```

How to rollout views?

https://github.com/mydataproject/pr/100

```
1   val joinedData = spark.table("T1").join(spark.table("T2"), spark.table("T1")("a") === spark.table("T2")("c"), "inner")
2     .join(spark.table("T3"), spark.table("T2")("b") === spark.table("T3")("a"), "left_outer")
3     .join(spark.table("T4"), spark.table("T3")("c") === spark.table("T4")("a"), "right_outer")
4
5   val result = joinedData
6     .filter($"T1.a" > 10 && $"T4.b" < 20)
7     .select($"T1.a", $"T2.b", $"T3.c", $"T4.d")
8     .groupBy($"T1.a", $"T2.b")
9     .agg(
10      max($"T3.c").as("max_c"),
11      min($"T4.d").as("min_d")
12    )
13    .orderBy($"T1.a".desc, $"T2.b".asc)
14
```

https://github.com/mydataproject/pr/100

```
1   val joinedData = spark.table("T1_UC1").join(spark.table("T2_UC1"), spark.table("T1_UC1")("a") === spark.table("T2_UC1")("c"), "inner")
2     .join(spark.table("T3_UC1"), spark.table("T2_UC1")("b") === spark.table("T3_UC1")("a"), "left_outer")
3     .join(spark.table("T4_UC1"), spark.table("T3_UC1")("c") === spark.table("T4_UC1")("a"), "right_outer")
4
5   val result = joinedData
6     .filter($"T1_UC1.a" > 10 && $"T4_UC1.b" < 20)
7     .select($"T1_UC1.a", $"T2_UC1.b", $"T3_UC1.c", $"T4_UC1.d")
8     .groupBy($"T1_UC1.a", $"T2_UC1.b")
9     .agg(
10      max($"T3_UC1.c").as("max_c"),
11      min($"T4_UC1.d").as("min_d")
12    )
13    .orderBy($"T1_UC1.a".desc, $"T2_UC1.b".asc)
14
```

How to roll out views?

Not user facing migration!

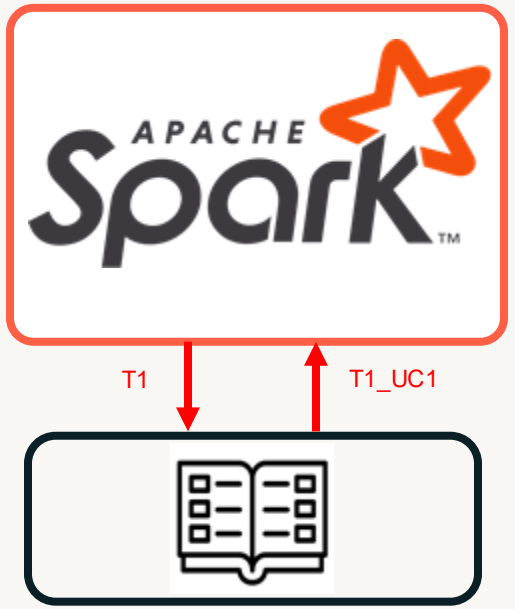
Large scale migration?

- Expensive & Slow
- Exposes context-specific view names
- Hard to evolve, include new policies
- Does not work for views

ViewShift

https://github.com/mydataproject/pr/100

```
1     val joinedData = spark.table("T1").join(spark.table("T2"), spark.table("T1")("a") === spark.table("T2")("c"), "inner")
2       .join(spark.table("T3"), spark.table("T2")("b") === spark.table("T3")("a"), "left_outer")
3       .join(spark.table("T4"), spark.table("T3")("c") === spark.table("T4")("a"), "right_outer")
4
5     val result = joinedData
6       .filter($"T1.a" > 10 && $"T4.b" < 20)
7       .select($"T1.a", $"T2.b", $"T3.c", $"T4.d")
8       .groupBy($"T1.a", $"T2.b")
9       .agg(
10        max($"T3.c").as("max_c"),
11        min($"T4.d").as("min_d")
12      )
13      .orderBy($"T1.a".desc, $"T2.b".asc)
14
```

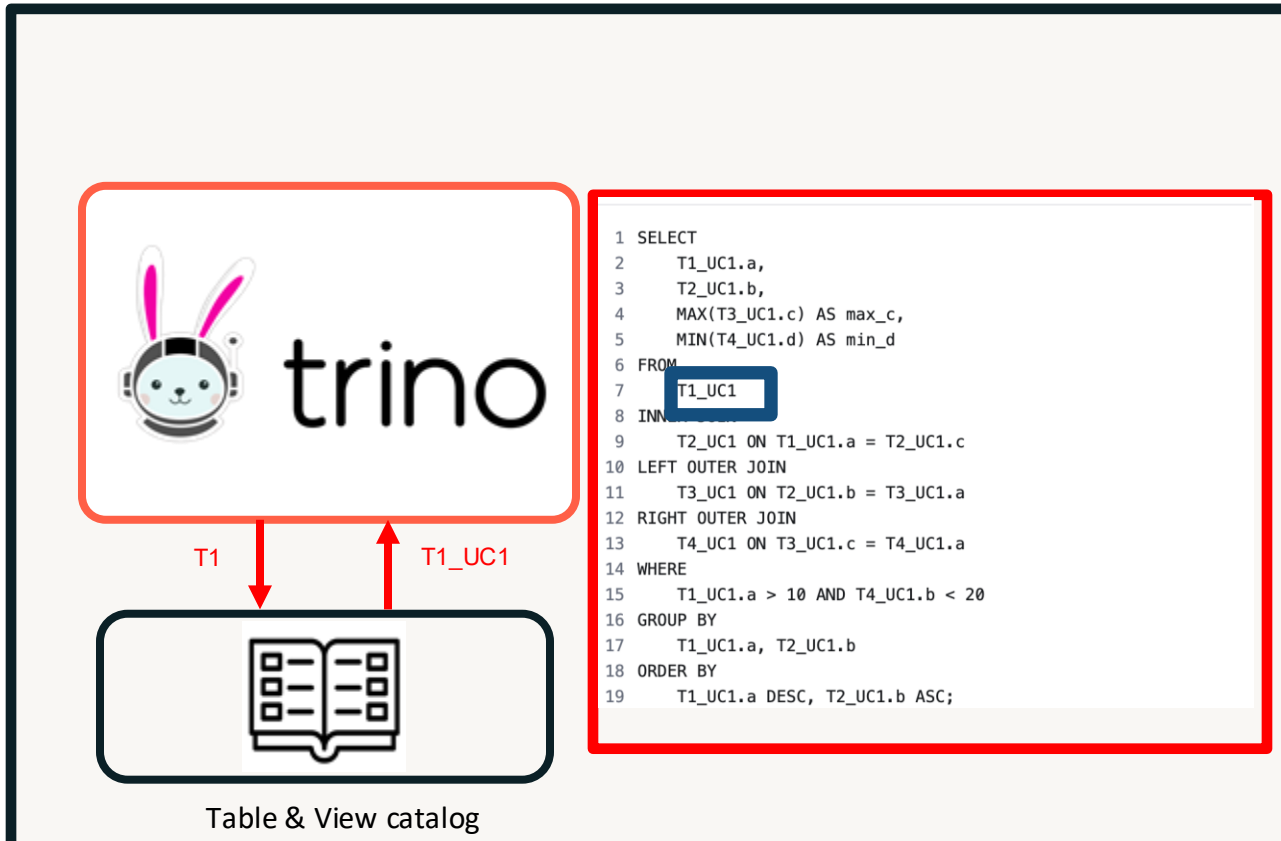



```
1     val joinedData = spark.table("T1_UC1").join(spark.table("T2_UC1"), spark.table("T1_UC1")("a"), spark.table("T2_UC1")("c"), "inner")
2       .join(spark.table("T3_UC1"), spark.table("T2_UC1")("b") === spark.table("T3_UC1")("a"), "left_outer")
3       .join(spark.table("T4_UC1"), spark.table("T3_UC1")("c") === spark.table("T4_UC1")("a"), "right_outer")
4
5     val result = joinedData
6       .filter($"T1_UC1.a" > 10 && $"T4_UC1.b" < 20)
7       .select($"T1_UC1.a", $"T2_UC1.b", $"T3_UC1.c", $"T4_UC1.d")
8       .groupBy($"T1_UC1.a", $"T2_UC1.b")
9       .agg(
10        max($"T3_UC1.c").as("max_c"),
11        min($"T4_UC1.d").as("min_d")
12      )
13      .orderBy($"T1_UC1.a".desc, $"T2_UC1.b".asc)
14
```

ViewShift

https://github.com/mydataproject/pr/100

```
SELECT
    T1.a,
    T2.b,
    MAX(T3.c) AS max_c,
    MIN(T4.d) AS min_d
FROM
    T1
INNER JOIN
    T2 ON T1.a = T2.c
LEFT OUTER JOIN
    T3 ON T2.b = T3.a
RIGHT OUTER JOIN
    T4 ON T3.c = T4.a
WHERE
    T1.a > 10 AND T4.b < 20
GROUP BY
    T1.a, T2.b
ORDER BY
    T1.a DESC, T2.b ASC;
```

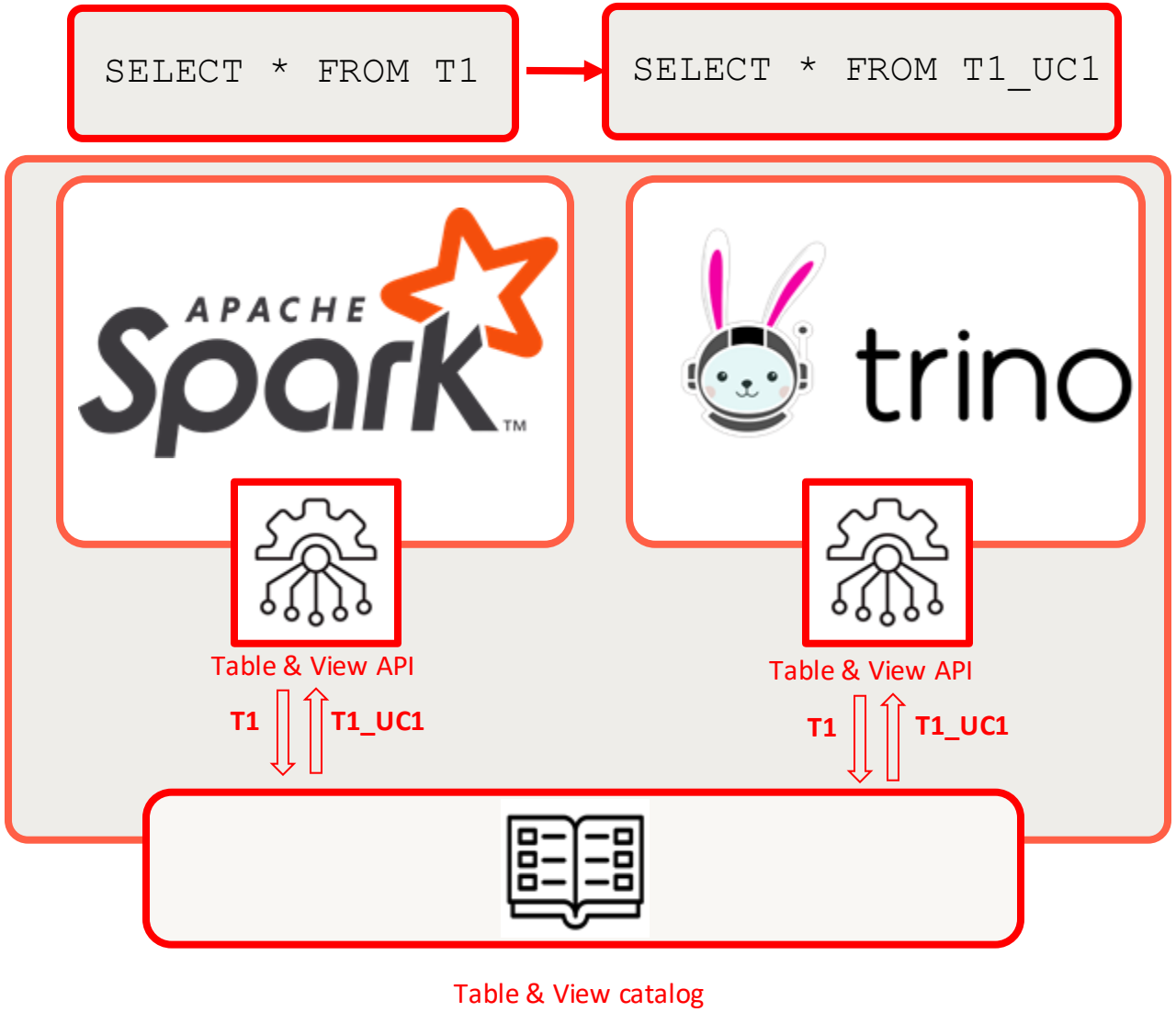



```
SELECT
    T1_UC1.a,
    T2_UC1.b,
    MAX(T3_UC1.c) AS max_c,
    MIN(T4_UC1.d) AS min_d
FROM
    T1_UC1
INNER JOIN
    T2_UC1 ON T1_UC1.a = T2_UC1.c
LEFT OUTER JOIN
    T3_UC1 ON T2_UC1.b = T3_UC1.a
RIGHT OUTER JOIN
    T4_UC1 ON T3_UC1.c = T4_UC1.a
WHERE
    T1_UC1.a > 10 AND T4_UC1.b < 20
GROUP BY
    T1_UC1.a, T2_UC1.b
ORDER BY
    T1_UC1.a DESC, T2_UC1.b ASC;
```

ViewShift: Benefits

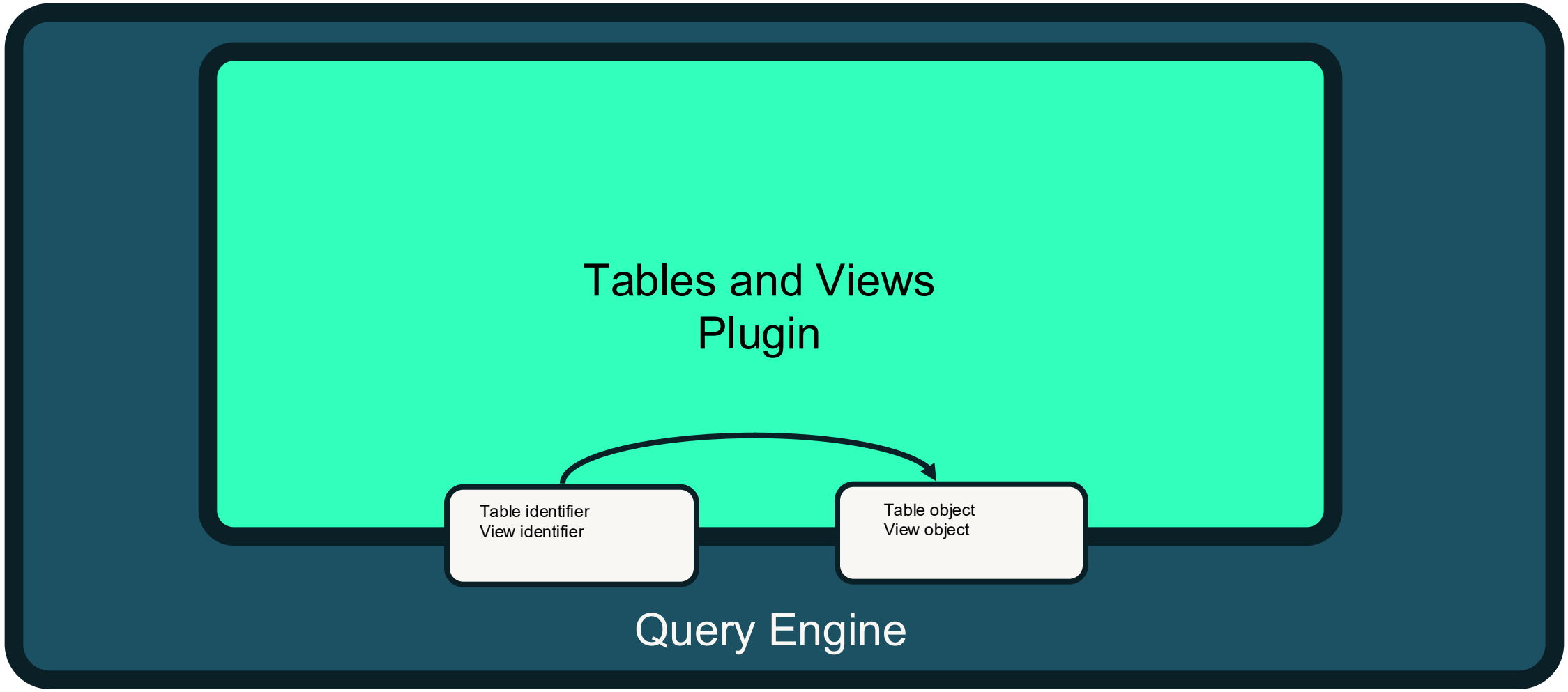
Dynamically route tables to views at runtime!

- Transparent
- Familiar names
- Works for next regulation
- Easy version management



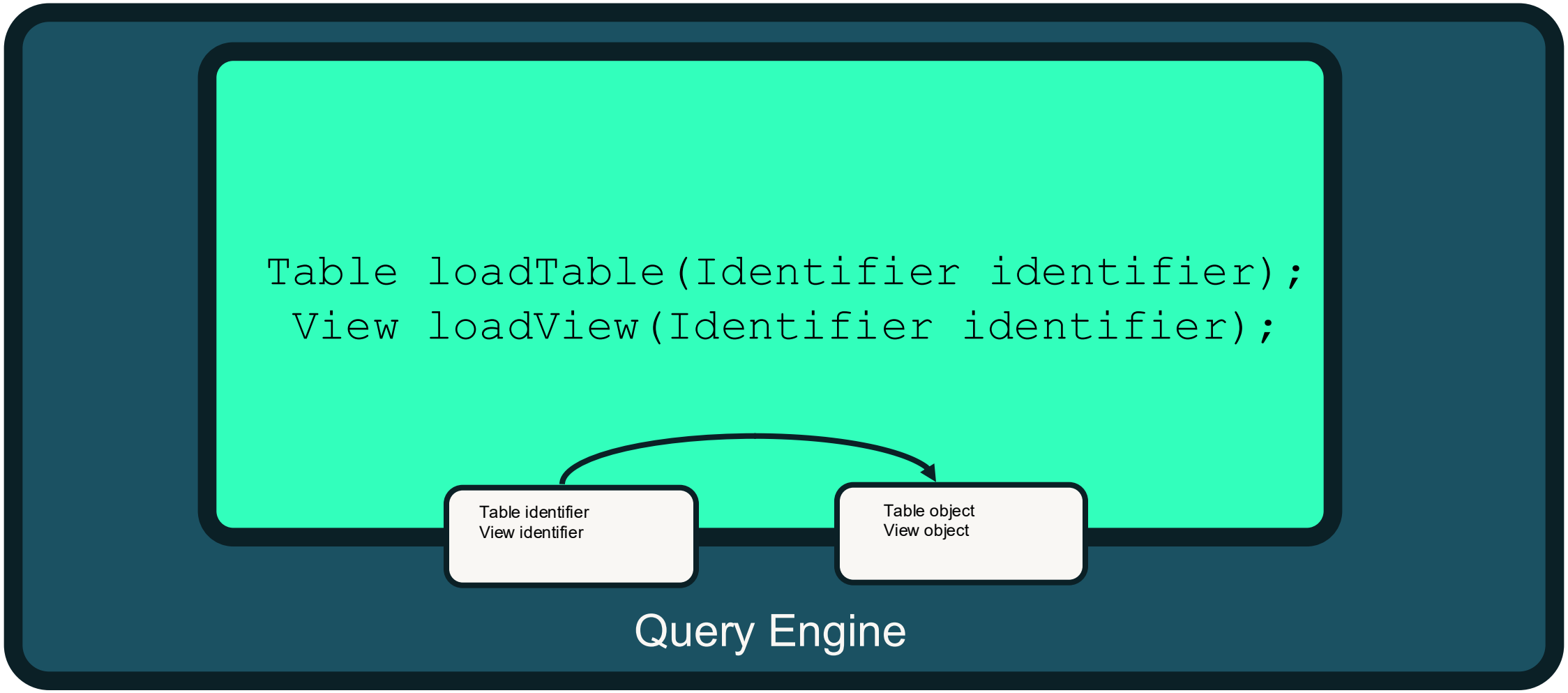
ViewShift: Architecture

Plugin within a plugin



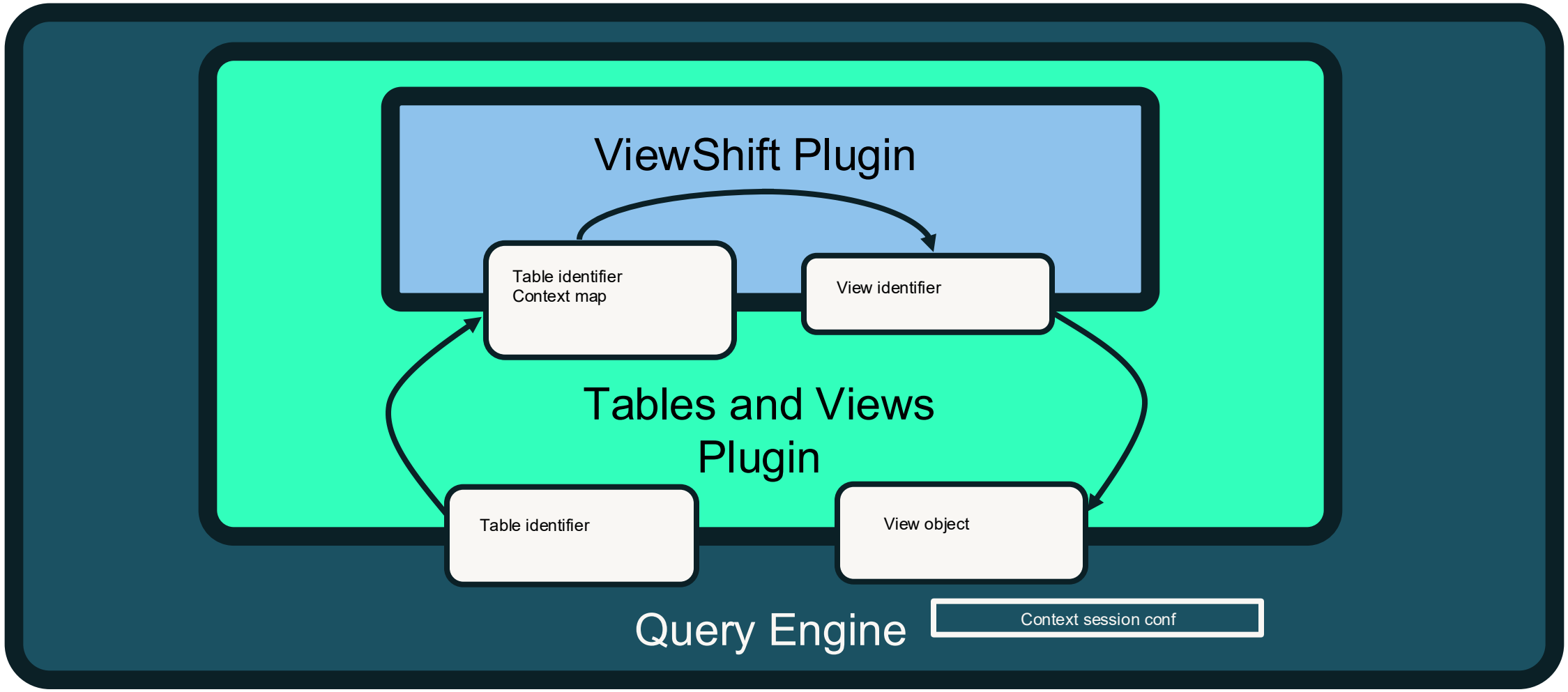
ViewShift: Architecture

Plugin within a plugin



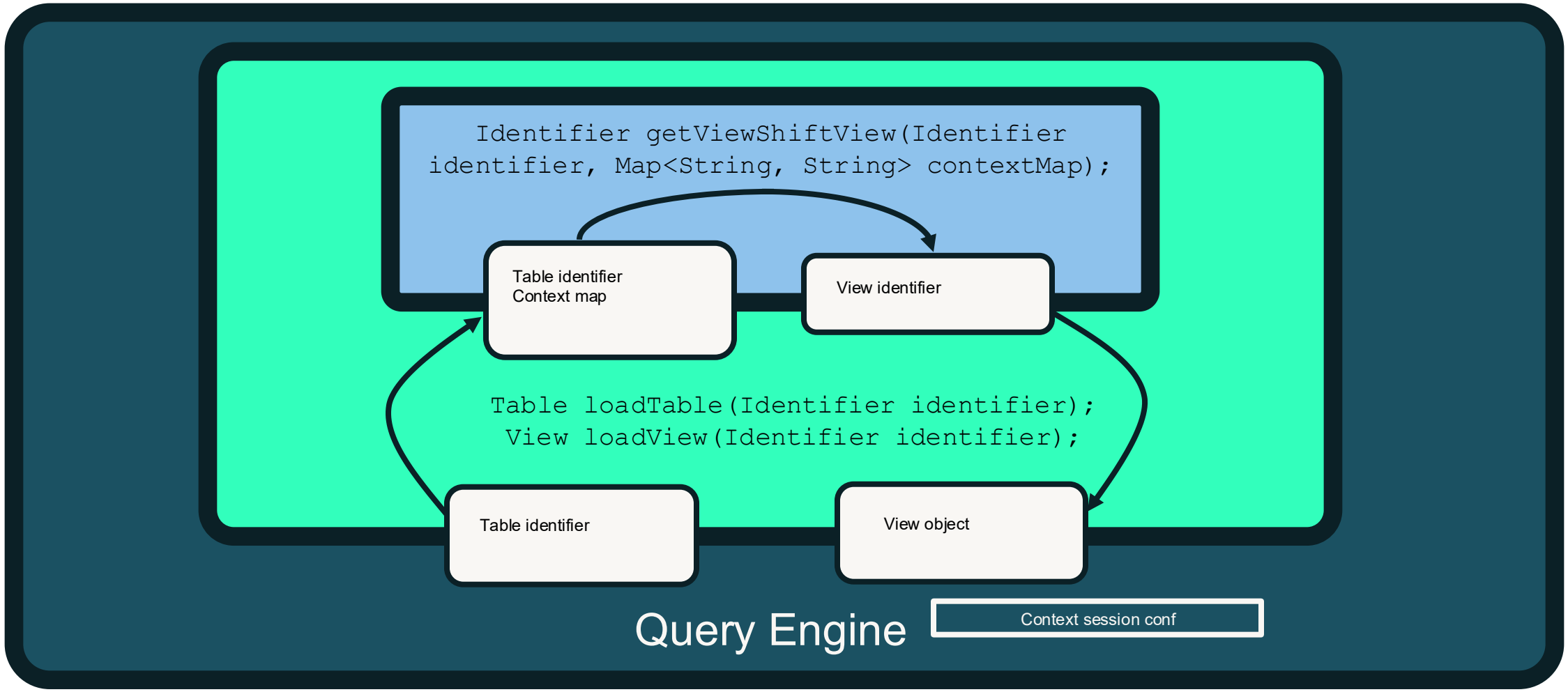
ViewShift: Architecture

Plugin within a plugin

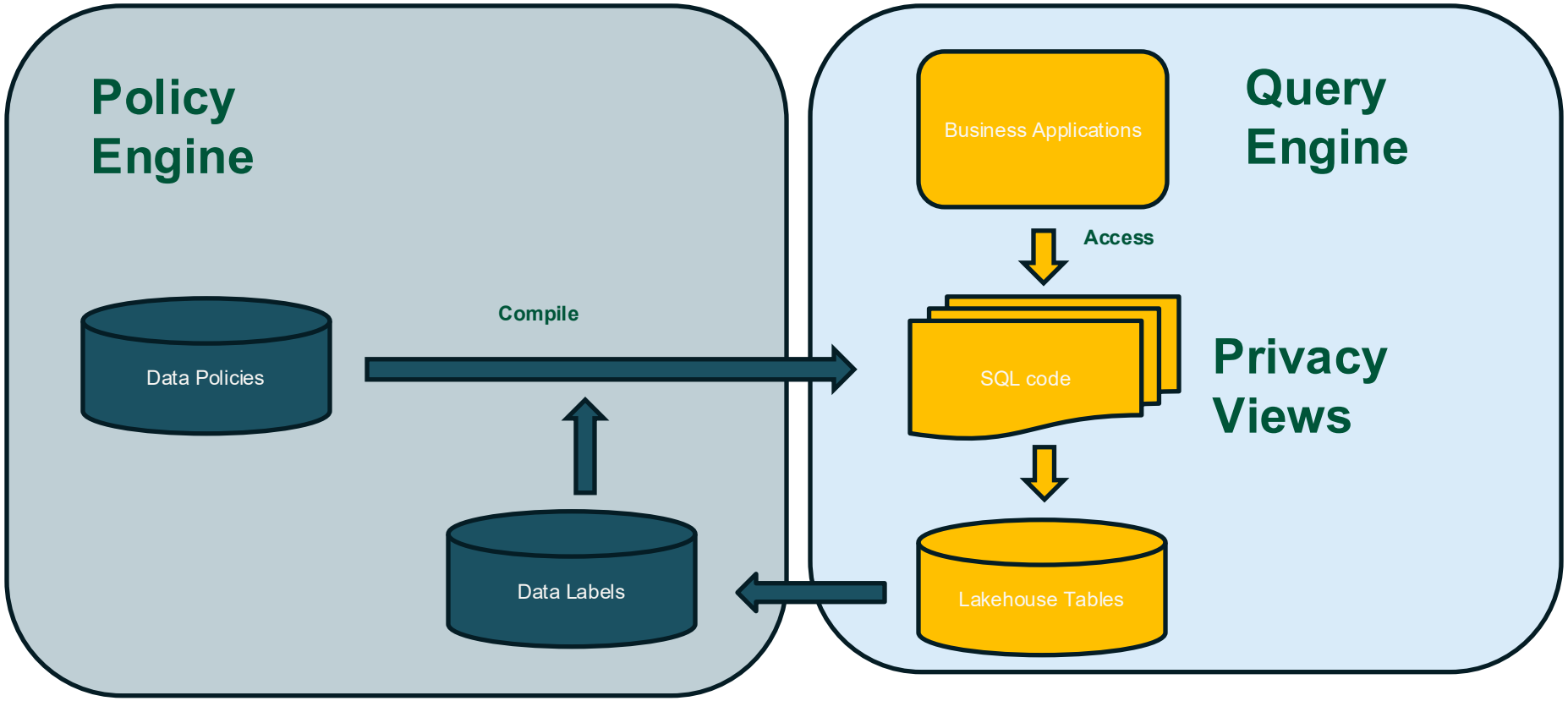


ViewShift: Architecture

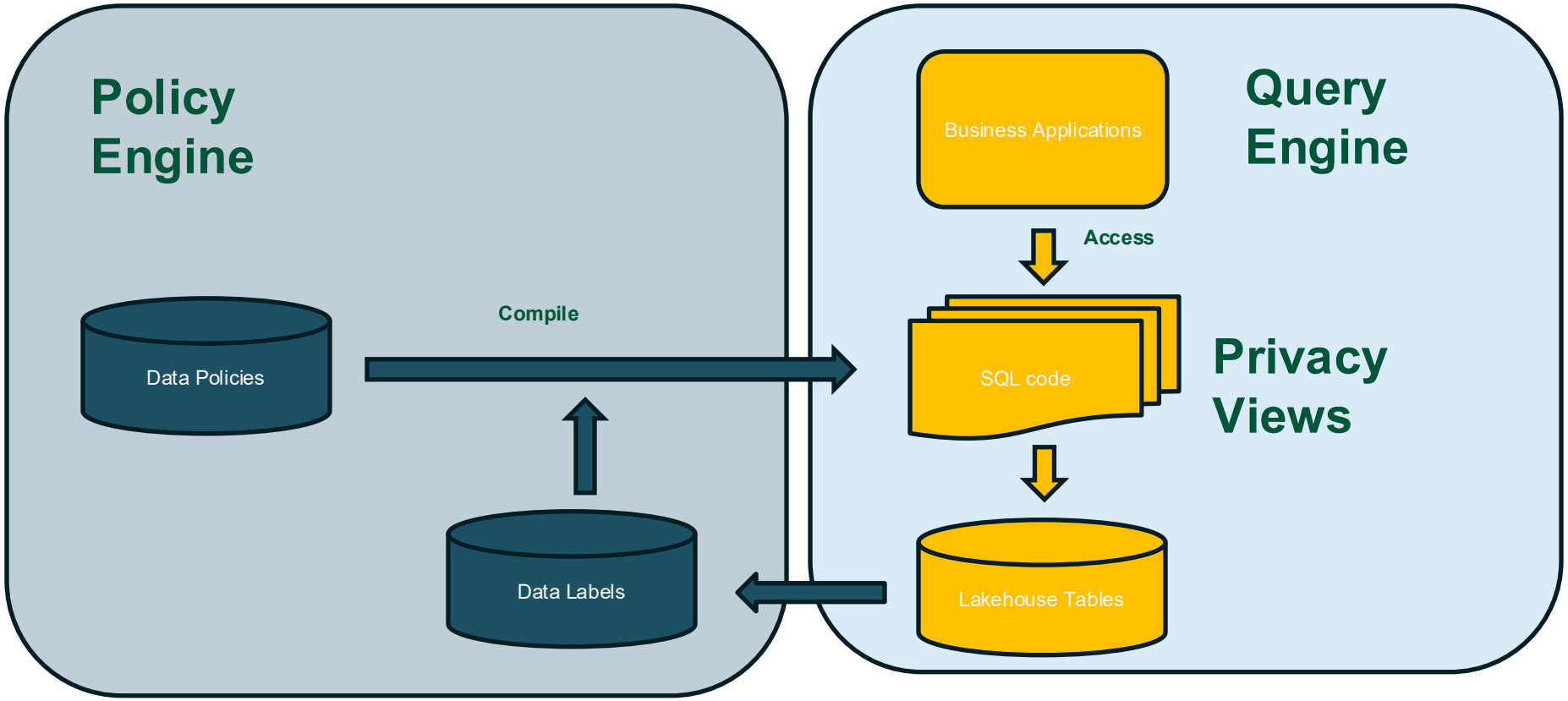
ViewShift: Architecture



The policy-based enforcement/masking system



The policy-based enforcement/masking system



Privacy View: SQL representation of **applicable policies** on a table access for a **given business purpose**

Example

memberId	yearBorn	gender
1	1991	F
2	1992	M

Demographic

memberId	adsAllowAge
1	False
2	True

Preferences

Example

Learning Application

```
SELECT memberId, yearBorn
FROM Demographic
```

`spark.sql.viewshift.enabled=true`

APACHE Spark

memberId	yearBorn
1	1991
2	1992

memberId	yearBorn	gender
1	1991	F
2	1992	M

Demographic

memberId	adsAllowAge
1	False
2	True

Preferences

Example

Learning Application

SELECT memberId, yearBorn
FROM Demographic

Ads
Application

spark.sql.viewshift.enabled=true

APACHE Spark

memberId	yearBorn
1	1991
2	1992

memberId	yearBorn
1	null
2	1992

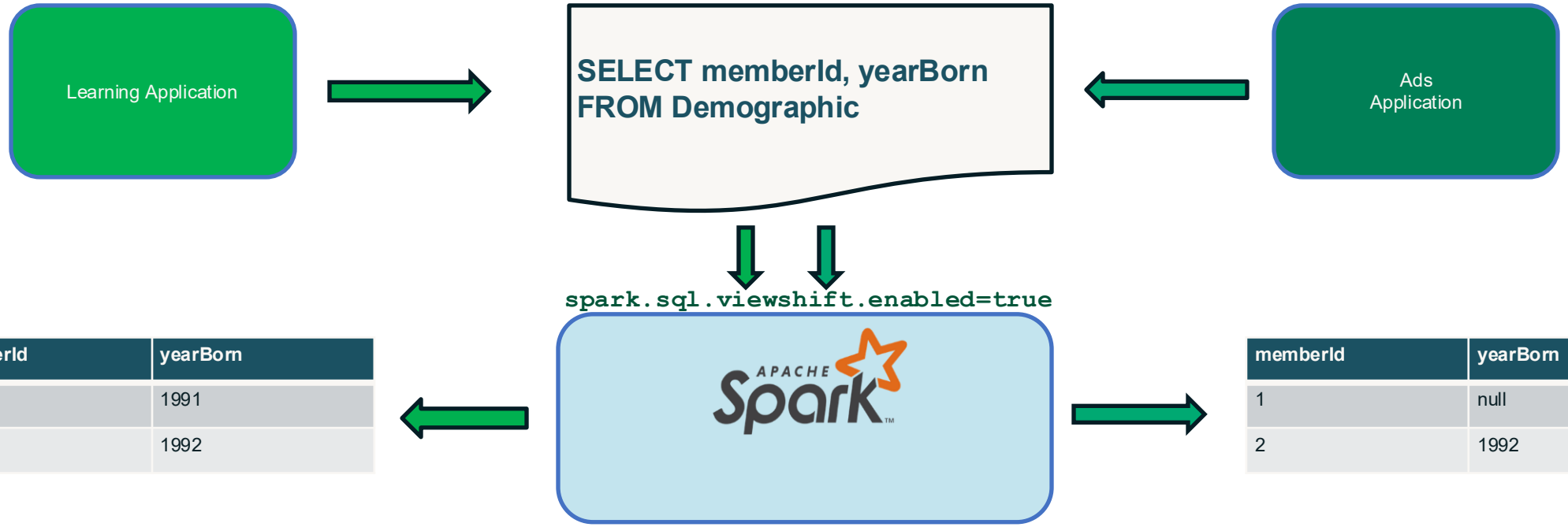
memberId	yearBorn	gender
1	1991	F
2	1992	M

Demographic

memberId	adsAllowAge
1	False
2	True

Preferences

Behind the scene



memberId	yearBorn
1	1991
2	1992

memberId	yearBorn
1	null
2	1992

memberId	yearBorn	gender
1	1991	F
2	1992	M

Behind the scene

Learning Application

Ads Application

SELECT memberId, yearBorn
FROM Demographic

`spark.sql.viewshift.enabled=true`

APACHE Spark

ViewShiftPlugin

memberId	yearBorn
1	1991
2	1992

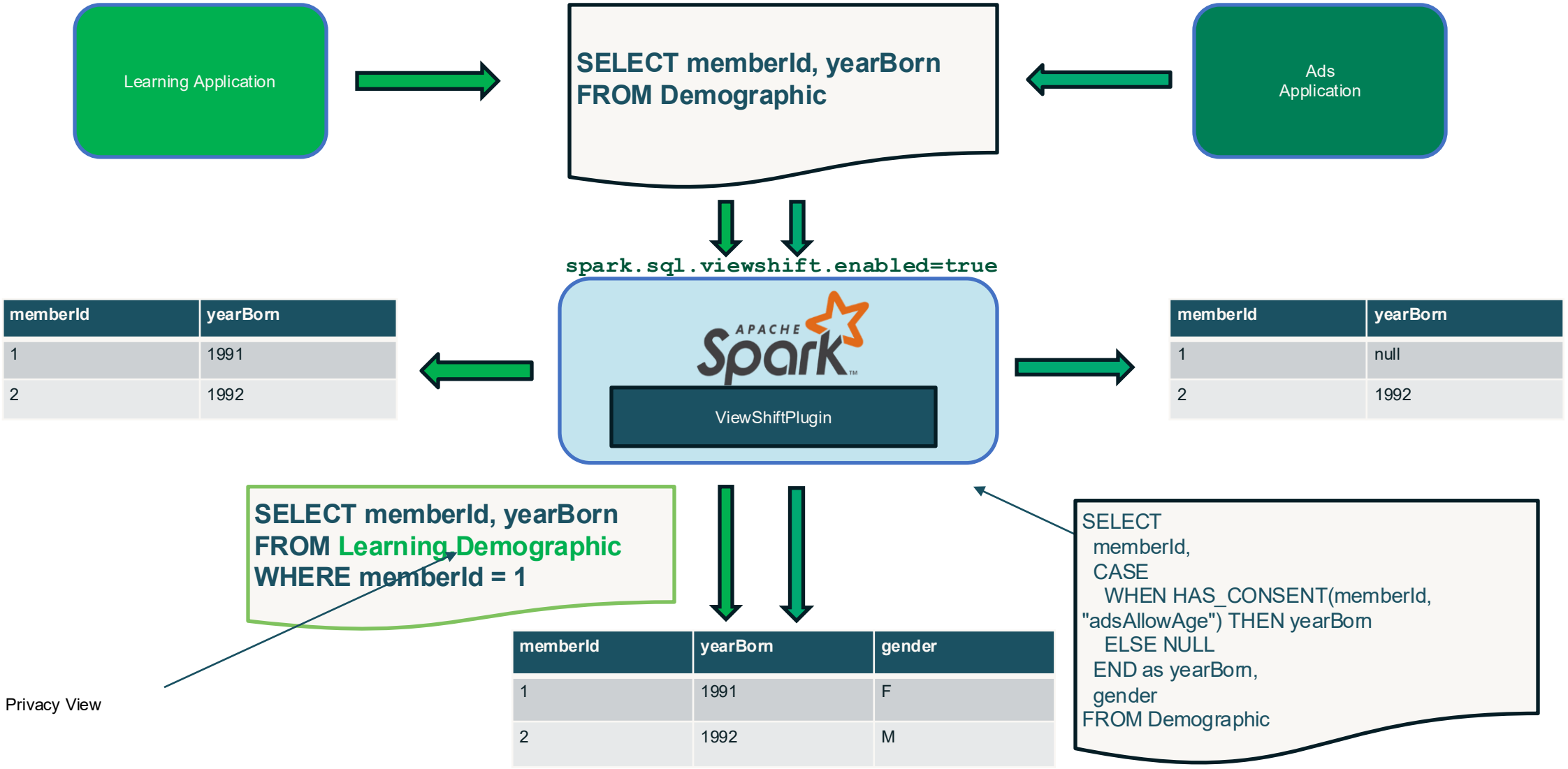
memberId	yearBorn
1	null
2	1992

SELECT memberId, yearBorn
FROM Learning.Demographic
WHERE memberId = 1

SELECT memberId, yearBorn
FROM Ads.Demographic
WHERE memberId = 1

memberId	yearBorn	gender
1	1991	F
2	1992	M

Behind the scene



Thank you!

Resources:

Full paper:

https://arxiv.org/abs/2502.01998

arXiv:2502.01998v1 [cs.DB] 4 Feb 2025

Data Guard: A Fine-grained Purpose-based Access Control System for Large Data Warehouses

Khai Tran*, Sudarshan Vasudevan*, Pratham Desai*, Alex Gorelik, Mayank Ahuja, Athrey Yadatore Venkateshababu, Mohit Verma, Dichao Hu, Walaa Eldin Moustafa, Vasanth Rajamani†‡, Ankit Gupta, Issac Buenrostro, Kalinda Raina
LinkedIn Coporation, OpenAI Inc†
{khtran,suvasudevan,padesai,agorelik,mahuja,avenkateshababu,moverma,dihu,wmoustafa}@linkedin.com
vasanth@openai.com,{aigupta,ibuenros,kraina}@linkedin.com

ABSTRACT

The last few years have witnessed a spate of data protection regulations in conjunction with an ever-growing appetite for data usage in large businesses, thus presenting significant challenges for businesses to maintain compliance. To address this conflict, we present *Data Guard* - a fine-grained, purpose-based access control system for large data warehouses. Data Guard enables authoring policies based on semantic descriptions of data and purpose of data access. Data Guard then translates these policies into SQL views that mask data from the underlying warehouse tables. At access time, Data Guard ensures compliance by transparently routing each table access to the appropriate data-masking view based on the purpose of the access, thus minimizing the effort of adopting Data Guard in existing applications. Our enforcement solution allows masking data at much finer granularities than what traditional solutions allow. In addition to row and column level data masking, Data Guard can mask data at the sub-cell level for columns with non-atomic data types such as structs, arrays, and maps. This fine-grained masking allows Data Guard to preserve data utility for consumers while ensuring compliance. We implemented a number of performance optimizations to minimize the overhead of data masking operations. We perform numerous experiments to identify the key factors that influence the data masking overhead and demonstrate the efficiency of our implementation.

PVLDB Reference Format:
Khai Tran, Sudarshan Vasudevan, Pratham Desai, Alex Gorelik, Mayank Ahuja, Athrey Yadatore Venkateshababu, Mohit Verma, Dichao Hu, Walaa Eldin Moustafa, Vasanth Rajamani, Ankit Gupta, Issac Buenrostro, Kalinda Raina. Data Guard: A Fine-grained Purpose-based Access Control System for Large Data Warehouses. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts are available at https://github.com/linkedin/dataguard-udfs.

*Equal contributors.
†Work done while the author was at LinkedIn Coporation.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

The last two decades have witnessed a tremendous growth both in the collection and usage of data to power use cases such as business analytics, recommender systems and personalization. A significant amount of data that businesses collect from end users to power these applications is sensitive or personally identifiable. Understandably, there have been concerns from regulators, privacy advocates and various other citizen groups alike around the lack of transparency in the data collection and data handling practices adopted by businesses. Regulations such as GDPR [23], CCPA [10], HIPAA [25], DMA [18] and EU AI Act [20] have been enacted into law in response to these privacy concerns and to drive transparency into the data collection and data usage practices of businesses. Specifically, these regulations require businesses to limit the usage of user data only for the purposes for which it is collected. As an example, the GDPR regulation requires businesses to seek consent from their end users (or formally, data subjects) in order to use sensitive attributes such as age and gender for advertising purposes. At the same time, the use of this data is often permitted in anonymized form to enable business use cases intended for demographic analysis.

To address these privacy concerns and ensure compliance with regulations, several solutions have been proposed [8, 13, 40] to enforce access control policies based on the purpose of data access. The systems are commonly referred to as *Purpose-based Access Control* (PBAC) systems. Depending on the granularity of enforcement, these systems can either authorize/deny access to an entire dataset (*coarse-grained* access control), or they can apply dynamic masking to hide contents of individual rows, columns or cells within the dataset (*fine-grained* access control). Enforcing policies dictated by these regulations across a large organization can be a challenging endeavor for the following reasons:

(1) A typical modern data stack involves multiple data processing systems such as Apache Spark [33], Trino [36] and Apache Flink [21]. Thus, any access control solution is required to apply data access policies consistently across a diversity of data processing systems used in an organization.
(2) Many data-centric organizations commonly support 10-100s of thousands of datasets, 10-100s of use cases (or *purposes*) and 100s-1000s of data pipelines accessing exabyte-scale data every day. A practical solution for access control must therefore be sufficiently scalable to meet these demands.

Coral Github:

https://github.com/linkedin/coral

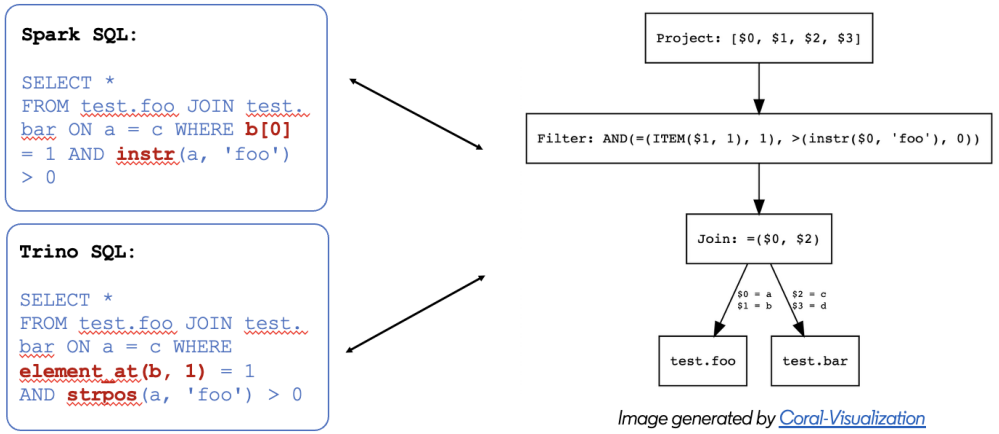


Image generated by Coral-Visualization

Policy Engine Example – Policy Matching

TableName	Field	Label
Demographic	memberId	KEY
Demographic	yearBorn	AGE
Demographic	gender	GENDER

Data Labels

```
Purpose: Ads
Label: AGE
Rule:
  if adsAllowAge:
    KEEP
  else:
    ERASE
```

AdsPolicyForAge

TableName	Purpose	ApplicablePolicies
Demographic	Ads	[{"Field":"yearBorn", "Policy":"AdsPolicyForAge"}]
Demographic	Learning	[]

Matching Table

Policy Engine Example – SQL compilati

Purpose: Ads
Label: AGE
Rule:
  if adsAllowAge:
    KEEP
  else:
    ERASE

Matching Table

TableName	Purpose	ApplicablePolicies
Demographic	Ads	[{"Field":"yearBorn", "Policy":"AdsPolicyForAge"}]
Demographic	Learning	[]

```
SELECT
  memberId,
  CASE
    WHEN HAS_CONSENT(memberId, "adsAllowAge") THEN yearBorn
    ELSE NULL
  END as yearBorn,
  gender
FROM Demographic
```

Ads.Demographic

```
SELECT *
FROM Demographic
```

Learning.Demographic

Policy Engine Example – SQL compilati

Purpose: Ads
Label: AGE
Rule:
  if adsAllowAge:
    KEEP
  else:
    ERASE

Matching Table

TableName	Purpose	ApplicablePolicies
Demographic	Ads	[{"Field":"yearBorn", "Policy":"AdsPolicyForAge"}]
Demographic	Learning	[]

```
SELECT
  memberId,
  CASE
    WHEN HAS_CONSENT(memberId, "adsAllowAge") THEN yearBorn
    ELSE NULL
  END as yearBorn,
  gender
FROM Demographic
```

Ads.Demographic

```
SELECT *
FROM Demographic
```

Learning.Demographic

HAS_CONSENT(memberId: BIGIN,
consentName: VAR CHAR):
Returns true iff memberId has consent on consentName

databricks

DATA+AI SUMMIT

Instructions

Read me first!

When using this template, create your new slides at the very top of the slide order. Explore the example slides below to find useful layouts and graphics to pull into your design.

Remember some rooms can have up to 1,000 seats, please make sure your slides are legible and easy to read. When your slide deck is complete, delete this slide.

Note: title slide, forward-looking statement, and survey reminder are all **required and should not be deleted**

databricks

DATA+AI SUMMIT

Forward-looking Statement

This presentation has been prepared for informational purposes only. The information set forth herein does not purport to be complete or contain all relevant information. Statements contained herein are made as of the date of this presentation unless stated otherwise.

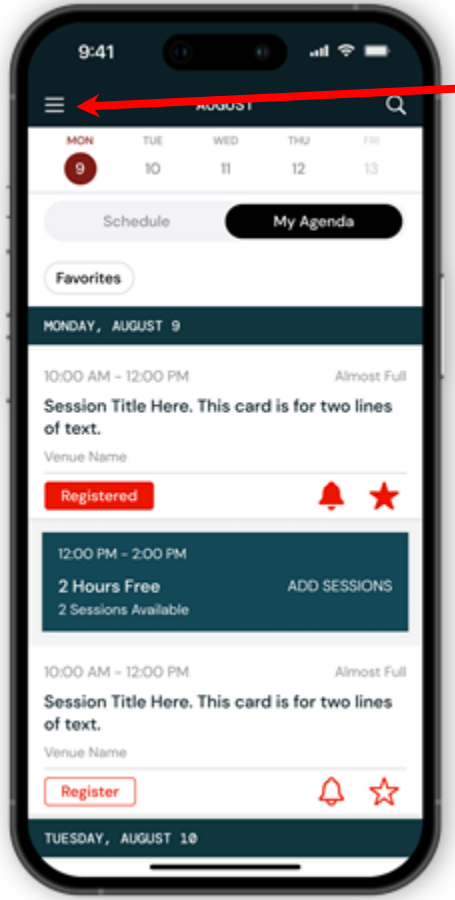
This presentation and the accompanying oral commentary may contain forward-looking statements. In some cases, forward-looking statements can be identified by terms such as “may”, “will”, “should”, “expects”, “plans”, “anticipates”, “could”, “intends”, “projects”, “believes”, “estimates”, “predicts”, or “continue”, or the negative of these words or other similar terms or expressions that concern Databricks’ expectations, strategy, plans, or intentions.

Forward-looking statements are based on information available at the time those statements are made and are inherently subject to risks and uncertainties that could cause actual results to differ materially from those expressed in or suggested by the forward-looking statements.

Forward-looking statements should not be read as a guarantee of future performance or outcomes. Except as required by law, Databricks does not undertake any obligation to publicly update or revise any forward-looking statement, whether as a result of new information, future developments or otherwise.

Complete Your Surveys

Your feedback has a direct impact on Data + AI Summit content



- You will receive a survey for each session attended
- Open the Databricks Events app and select “My Surveys” from the menu
- Surveys can also be submitted in the Attendee Portal

ViewShift: Hassle-Free Dynamic Policy Enforcement for Every Data Lake

Walaa Eldin Moustafa
June 10, 2025

THIS IS THE SUBHEAD

Section Title

THIS IS THE SUBHEAD

Section Title

Two-column content

Your subtitle here

Here's a column subheader

Your content goes here.

- Default bulleted list style
- Default bulleted list style
- Default bulleted list style

Here's a column subheader

Your content goes here.

- Default bulleted list style
- Default bulleted list style

Two column with icon space

Your subtitle here

These columns have space above for an icon

Your content goes here.

- Default bulleted list style
- Default bulleted list style
- Default bulleted list style

These columns have space above for an icon

Your content goes here.

- Default bulleted list style
- Default bulleted list style
- Default bulleted list style

Three-column text

Your subtitle here

Here's a column subheader

Your content goes here.

- Default bulleted list style
- Default bulleted list style
- Default bulleted list style

Here's a column subheader

Your content goes here.

- Default bulleted list style
- Default bulleted list style
- Default bulleted list style

Here's a column subheader

Your content goes here.

- Default bulleted list style
- Default bulleted list style
- Default bulleted list style

Three-column with icon space

Your subtitle here

These columns have space for an icon

Your content goes here.

- Default bulleted list style
- Default bulleted list style
- Default bulleted list style

These columns have space for an icon

Your content goes here.

- Default bulleted list style
- Default bulleted list style
- Default bulleted list style

These columns have space for an icon

Your content goes here.

- Default bulleted list style
- Default bulleted list style
- Default bulleted list style

This is where you can add a short quote. Try to keep it to three lines max when possible.

This is where you can add a short quote. Try to keep it to three lines max when possible.

“The advance of technology is based on making it fit in so that you don’t really even notice it, so it’s part of everyday life.”

—Bill Gates, Co-founder of Microsoft

Three column card

Your subtitle here

Card number one

These cards are great for showing a 3-stage process, comparisons between 3 things, or whatever else you like

Card number two

These cards are great for showing a 3-stage process, comparisons between 3 things, or whatever else you like

Card number three

These cards are great for showing a 3-stage process, comparisons between 3 things, or whatever else you like

Righthand card

Your subtitle here

Add your content here.

- Bullet one
- Bullet two
- Bullet three

This spot is great for a diagram, image, small table, code snippet or screenshot

Lefthand card

Your subtitle here

This spot is great for a diagram, image, small table, code snippet or screenshot

Add your content here

- Bullet one
- Bullet two
- Bullet three

Full width card

Your subtitle here

This card is good for larger diagrams, tables or other general use

For more guidance, reference the assets linked below

Resource deck

databricks